COMPUTER ENGINEERING

QUEZON CITY

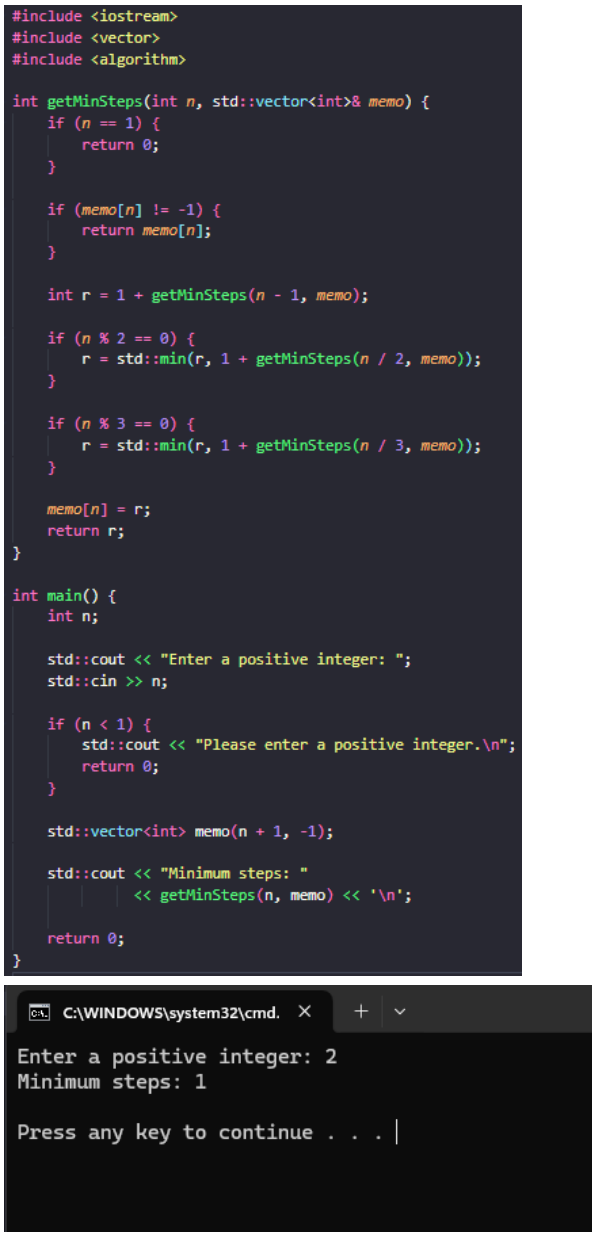
| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE 010 - Data Structures and Algorithm | **SECTION:** | CPE21S1 |
| **DATE PERFORMED:** | 10 - 13 - 2026 | **DATE SUBMITTED:** | 10 - 14 - 2026 |
| **NAME:** | Canoy Hail B.<br>Mauro, Ryan Lewis B. | **INSTRUCTOR:** | Engr. Jimlord M. Quejado |

## ACTIVITY NO. 12.1 [Algorithm Strategies]

### 1. PROCEDURE OUTPUT

| Strategy | Algorithm | Analysis |
|---|---|---|
| Recursion | **Recursive power** | **The function calls itself to calculate the power until it reaches the base.** |
| Brute Force | **Linear Search** | **The function calls all the elements manually one by one until the target element is found.** |
| Backtracking | **Depth First Search** | **Explores a path as far as possible and goes back when it can't continue the path.** |
| Greedy | **Selection Sort** | **The function chooses the smallest number and places it in the correct position.** |
| Divide-and-Conquer | **Merge Sort** | **The function divides and sorts them then finally merges it into sorted elements.** |

**Table 12-1. Algorithmic Strategies and Examples**

| | |
|---|---|
| **Screenshot** |  |
| **Analysis** | The memoization code uses recursion to find the minimum number of steps needed to reduce a positive integer to 1. It checks the possible operations, including subtracting 1 and dividing by 2 or 3 when divisible, then chooses the minimum number of steps. It stores the calculated results in the memoization array to avoid repeating the same calculations. This makes the algorithm more efficient than using plain recursion. |

```cpp
#include <iostream>
#include <vector>
#include <algorithm>

int getMinSteps(int n, std::vector<int>& memo) {
    if (n == 1) {
        return 0;
    }

    if (memo[n] != -1) {
        return memo[n];
    }

    int r = 1 + getMinSteps(n - 1, memo);

    if (n % 2 == 0) {
        r = std::min(r, 1 + getMinSteps(n / 2, memo));
    }

    if (n % 3 == 0) {
        r = std::min(r, 1 + getMinSteps(n / 3, memo));
    }

    memo[n] = r;
    return r;
}

int main() {
    int n;

    std::cout << "Enter a positive integer: ";
    std::cin >> n;

    if (n < 1) {
        std::cout << "Please enter a positive integer.\n";
        return 0;
    }

    std::vector<int> memo(n + 1, -1);

    std::cout << "Minimum steps: "
              << getMinSteps(n, memo) << '\n';

    return 0;
}
```

```
C:\WINDOWS\system32\cmd.
Enter a positive integer: 2
Minimum steps: 1

Press any key to continue . . .
```

**Table 12-2. Memoization Implementation**

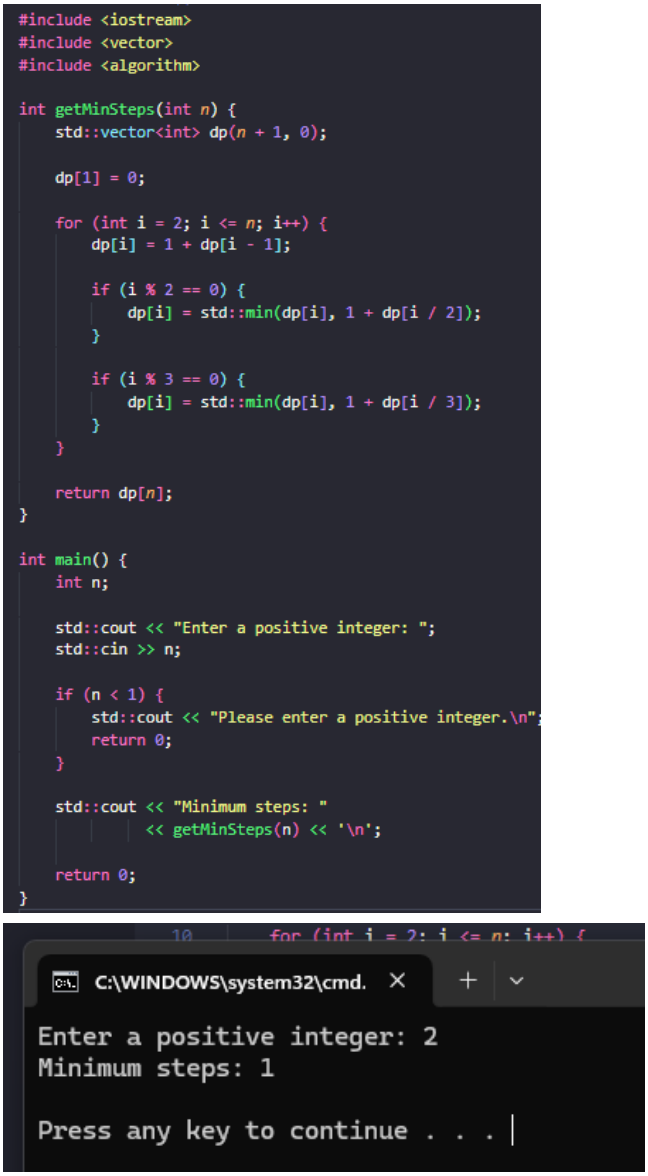
| | |
|---|---|
| **Screenshot** |  |
| **Analysis** | The bottom-up dynamic programming code uses a loop to calculate the minimum number of steps for each integer from 2 up to the given input. It uses previously calculated values stored in the DP array to determine the minimum steps for the current number. By comparing the possible operations, it selects the smallest result and stores it for future calculations. This method avoids recursive function calls while maintaining O(n)O(n) time and O(n)O(n) space complexity. |



**Table 12-3. Bottom-Up Dynamic Programming Implementation**

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

**Pseudocode**

```
START

INPUT matrix
INPUT targetCost

FUNCTION countPaths(row, col, cost)

 cost = cost + matrix[row][col]

 IF cost > targetCost
 RETURN 0
 END IF

 IF destination is reached
 IF cost = targetCost
 RETURN 1
 ELSE
 RETURN 0
 END IF
 END IF

 right = count paths by moving right
 down = count paths by moving down

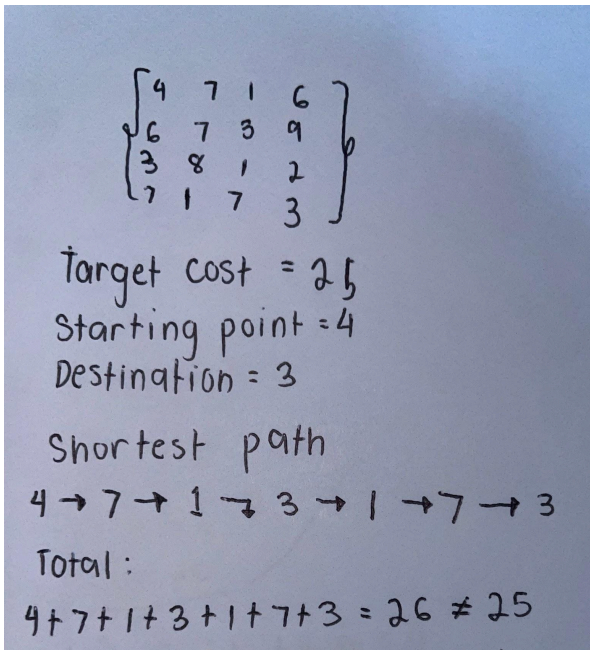
 RETURN right + down

END FUNCTION

DISPLAY countPaths(0, 0, 0)

END
```

**Solution By hand**




Since The shortest path is more than 25 the other path will also be more than 25 that is why there's no path that the exact cost is 25.

## Working C++ Code

main.cpp > ...

```cpp
#include <iostream>
using namespace std;

//To add the 4 by 4 matrix with its values
int matrix[4][4] = {
    {4, 7, 1, 6},
    {6, 7, 3, 9},
    {3, 8, 1, 2},
    {7, 1, 7, 3}
};

//To set the target Cost
int targetCost = 25;

//To create the function
int countPaths(int row, int col, int cost) {
    cost += matrix[row][col];

    //To stop if the cost is more than 25
    if (cost > targetCost) {
        return 0;
    }

    //To check if it already reached 3 in 4th row and 4th column
    if (row == 3 && col == 3) {
        if (cost == targetCost) {
            return 1;
        }
        return 0;
    }

    //to set the value of the movement
    int right = 0;
    int down = 0;

    //to check if its posible to go right
    if (col < 3) {
        right = countPaths(row, col + 1, cost);
    }

    //to move down if the value is less than 3
    if (row < 3) {
        down = countPaths(row + 1, col, cost);
    }

    //To sum up the total cost
    return right + down;
}

int main() {
    //To display the matrix
    cout << "Matrix:" << endl;
```

```
53
54      for (int i = 0; i < 4; i++) {
55          for (int j = 0; j < 4; j++) {
56              cout << matrix[i][j] << "\t";
57          }
58          cout << endl;
59      }
60
61      //To Display and label the answers
62      cout << "\nTarget Cost: " << targetCost << endl;
63
64      int result = countPaths(0, 0, 0);
65
66      cout << "Number of valid paths: " << result << endl;
67
68      return 0;
69  }
70
```

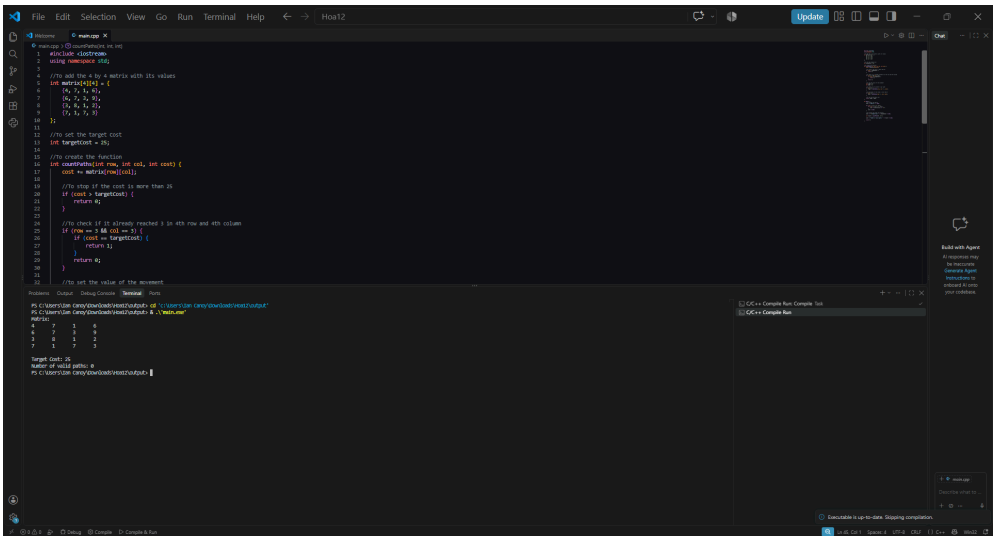
**Analysis of the Working Code**

- Basically the code uses the 4 x 4 matrix to find the number path with the total cost of 25. It starts with the 4 or the top left of the cell and it can move right or down until it reaches the number 3 at the bottom right cell. The countPaths function is just to check possible paths and adds the values of the cells visited and if the total cost exceeds 25 the path is path and if the program richest the final destination with the total cost of the 25 it will count as valid but here it's not because the shortest path is 26.
  The variables right and down store the number of the valid paths from each direction. The program then adds these values to get the total number of valid paths and after that we get the main function with the for loop. It basically prints all the values needed in the matrix then the printing of labels and answers.

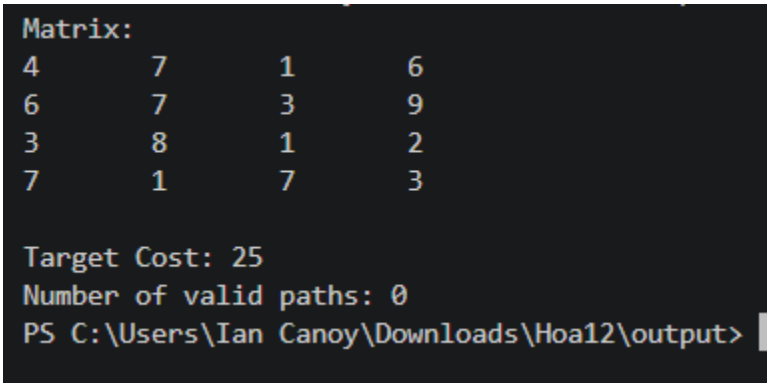
  The time complexity here is O(2^(M+N)) in the worst case because the function explores different combinations of the right and down movements.

  The Space complexity here is O(M+N) for the recursion stack.

## Screenshot of the source code




## Screenshot of the output



| Conclusion (Canoy Hail B.) |
| --- |
| - In this activity I learned about the different algorithmic strategies and how they are used to solve problems. I learned how recursion works and other things. In this activity I also learned and understood how memoization and dynamic programming can make programs more efficient just by avoiding the repeated calculations. |
| **Conclusion (Mauro, Ryan Lewis B)** |
| - In this activity I gained a better understanding about how algorithms work and how they can be applied to a lot of different programming problems. I also learned how to follow the possible movements in the matrix and to determine which paths meet the required cost. I also learned to save time using techniques like memoization and dynamic programming. |

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **none**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **none**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **none**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.